Introduction

Cool. Hello, everyone. And welcome to Lecture 5 of CME295. So first of all, thank you all for taking the time to take the midterm last week. So I hope it was reasonable for you. So for those of you who are auditing, in case you are interested in taking the exam, just know that the exam and the solutions are both posted on the website. And so now, you know a little bit how the exam looks like. So for the final, we'll have the same format, except that the content will be on lectures 5, so this one, up until 9. So all good on that point? Cool. Great. So with that, we're going to start the lecture. So today, we're going to talk about LLM tuning. So as usual, we're just going to recap what we saw last time. So last time was already two weeks ago. But we talked about how to train an LLM. And in particular, we've looked at two important steps. So the first one was called pre-training, where you're basically taking a model that has been initialized, and you're trying to teach the model about language, about code. So you have this step that is very time consuming, expensive, compute heavy, that is happening on a lot of data. So we've seen training optimizations on how to make that happen. So we've seen techniques to parallelize that across GPUs. So we've seen data parallelism methods and, in particular, 0. So the variant 0, 1, 2, 3. And we've also seen very quickly what model parallelism was in this case. So at the end of this step, what you obtain is a model that knows about the structure of language, about codes-- basically, all the texts that it has been fed. But what this model can do is only predict the next token. So it's a great autocompleter, but it's not a helpful model yet, which is why we have a second step. And we saw here, it's typically called finetuning or SFT for Supervised Fine Tuning. So here what we do is we take our pre-trained model, and we train it for specific tasks. So nowadays, you have ChatGPT and all these chat assistants. So this can be one application. So transforming your model into an assistant. And so here, the goal is to teach the model how to behave. So the model already knows what language is, what code is, et cetera. And you're just trying to make it behave like the use case that you're trying to tune it for. So typically here, what you have is a data set that is much smaller in scale, but of much higher quality. And you're basically taking your model, your pre-trained model, and teaching it exactly which tokens to predict with the next token

prediction task. And we've seen, LoRA, which is a parameter efficient method, which does not tune all the weights, but in a clever way, introduces low rank matrices that are the ones that are being tuned. And we had stopped there. So what we're going to see today is how to align the model to align with, what we call, human preferences. And so here, we're taking our model that has been finetuned for a specific task. And trying to align our model to make it more something that a human would like or that some metric that we're defining would be more aligned with that. So as an example, let's suppose if you have an assistant at the end step 2. So it's very much possible that your assistant is behaving the way you want, but not, let's say, at the tone that you want. So it's not, let's say, friendly or it's not safe. So you want to tune those aspects in that third step.

Preference tuning

Cool. So this step is called preference tuning. And we're going to exactly see what that is. So here for context, let's suppose that we have an SFT model. And so by SFT model, I mean a model that has gone through the pre-trained stage and the finetuning stage. And so for instance, we may ask our model to suggest a new activity we could do with our teddy bear. And here the model, let's say, responds with, "I would suggest you to not spend much time with your teddy bear at all." So this is the response of an assistant, but it's not necessarily aligned with what we want. So the idea here is to take these, quote unquote, "bad outputs" and find an output or rewrite an output that we would want to have instead. So this pair would be what we call a preference pair. So in other words, given this prompt, we have two responses-- one that we want to see, which is this one. And I'm going to read it in a second. And then the other one that we do not want to see. So for instance, in this example, the answer we want to see is, "Of course, teddy bears not only make awesome companions for a delightful sleep, but can also be great buddies for fun activities," and just like suggest some activities. Does the setup sound good? Yeah? So long story short, we want to align our model with human preferences. So you may ask, we have this finetuning step already. Why would we want to have a third step? Well, during the second step, which is the finetuning stage, if you remember, what we did was construct a very high quality data set of the

kinds of prompts on which we want our model to behave in a certain way. So in order to compose such a data set, it's actually something that is very time consuming and very actually difficult, because the data set must be of very high quality. And in this case, what we're doing is not really teaching the model exactly what it should generate, but rather telling the model what kind of output it should prefer. So we're less in "please generate that" kind of thing and more a "I prefer this option" kind of thing. And so typically, if we asked you write a poem, write a great poem from scratch, it would typically be much more difficult to do rather than just showing you two poems, one bad poem and one great poem, and ask you to just say which one is better. So in order to obtain the data sets, it's already much easier. The second reason is-- so during the SFT stage, when we compose our data set of very high quality, there is one aspect that we really try to get right, which is the distribution of prompts. And what I mean by that is if we have too much of a given of prompt, our model will be more biased towards responding in that particular way. So what people try to do is to be careful about the distribution of prompts that are in the SFT data. And so here, let's say if our model misbehaves, if we were thinking about just adding one example in the SFT data set, well, we would have to be very careful about which prompt we're adding and whether it's not going to bias the model like too much in that direction. So that's the second reason. And then, the third reason is what I mentioned, which is the SFT data is typically very high quality. So if you're looking at all the missteps that your model is doing and trying to put that in the SFT data, you'll just have a hard time. It takes a lot of time. But one note here is if your SFT data is misbehaving a lot, it may also be due to the fact that your SFT data set has some problem. So preference tuning is not the answer to everything. Maybe it's better sometimes to just check your SFT data set for some issues. Does that sound good? Yeah. [AUDIO OUT] Yeah, so to do that in the preference tuning stage? OK, so the question is, we saw LoRA for SFT, what is the equivalent for preference tuning? So we'll see that later in the lecture. But you can think of LoRA as being some way to, I guess, reduce the number of parameters that you need to tune. That is slightly different with the objective function that you're using to train your model. And here, preference tuning, you can think of it more as a different objective function, but it may very well be something that you also use LoRA for. So the two are not incompatible. But it will become more clear later on. But yeah, great question. One last thing I

will add here is another difference with the SFT stage is that preference tuning allows us to inject some negative signal. Because SFT is all about teaching the model about what it should predict, but it does not teach the model about what it should not predict. And we will see that preference tuning allows you to inject some negative signal.

Data collection

Cool. So to start, of course, we need to have our preference pairs. And so we're going to look at this data collection step first. So here's the setup. You have a prompt. Let's say write a poem. And you have a given response, which is the poem that is being generated by the model. You have a few ways to construct your preference data. So either you start with a pointwise mindset, where you score each proposed poem with some kind of pointwise score, and here pointwise score means a score just relative to one observation. You could very well do that, but I will say it's kind of hard. It's kind of tough as a human to say, OK, this one is, I don't know, a 0.9. This one is like a 0.2. It's not super clear exactly how you would scale that. The second idea is for you to get two observations at the time and for you to say which one is better. So this one is called pairwise preference data. And it's much easier. And then, the third one is listwise, which is get a list of, let's say, n poems, and then you just rank which one is best, which one is worst, et cetera, et cetera. And I guess this one is easier than pointwise, because you don't have to specify, I guess, how much better it is, but I guess it's still a bit more complicated. And that's the reason why people typically use pairwise. So what they do is they collect pairwise preference data. Meaning for each prompt, they have two possible answers, and then they just specify which one is better. And so that's the one we'll continue this lecture with. OK, so now you may ask, OK, great, pairwise preference data. But how do you get it? Well, here is the recipe. So in order to generate a pair of responses-- so first, you need a prompt. And we've seen-- previously, I think it's in lecture probably 3-- that what you could do was to generate different answers if you have a temperature that's positive. So typically, what people do is they put this prompt, let's say, twice into the model with a positive temperature, and then they can get two different answers. The prompt is typically something where we wanted to follow the distribution of

what the users typically ask. So the prompt x can be something that we obtain from the logs or from, let's say, a desired set of prompts. And then what we do is we have these two observations. So the first one is the prompt and the first response. The second one is the prompt and the second response. And what we do is we rate-- we compare them. So we can compare them with, of course, human ratings. But we can also compare them with some other metrics. So I'll just list a few. So we have LLM as a judge, which you may have heard of, which we have not seen yet, which we will see in a few lectures that is also typically used to just compare, I guess, how much better one observation is compared to another. We can also use some other metrics, rule-based one like BLEU, ROUGE, et cetera, although, it is not as used these days. And the simplest way to compare these two observations is to have a binary setting, where you say, OK, is response 1 better or worse than response 2? But you could also think of a more nuanced scale, meaning you can also say, OK, response 1 is much better, better, slightly better, slightly worse, worse, or much worse. So this is also something that you can do. But there are some challenges with that approach. Because for instance, if you, I don't know, consider human ratings, a lot of tasks are a little bit subjective. So in a lot of cases, actually, what people do is having a pairwise preference data set on the binary scale. So only is it better or worse. Does that sound good? Yeah. OK, so another way to obtain that data is to have-- to find in your logs a response that you did not like to take that response and to rewrite it, which is basically what we did here. So here, when we have the response here, what we did is take the response and rewrite a good one. So this is also what people do. But of course, it is a bit more involved, because you need to generate. And I told you that generation was kind of costly and tough, but this is also possible. Does the data collection make sense?

RLHF overview

Yeah? Cool. OK, so now, we have our preference data. And what we want is to align our model to prefer responses that were preferred by the rating, and, I guess, down weight the responses that were not preferred. And so in order to do that, we will see a method that's called RLHF. RLHF. And I'll see-- we'll see that in more details in a second. But as the name indicates, RLHF relies on RL.

So I'm just going to start with some RL basics. So do we have any RL experts here? Yeah? No? [CHUCKLES] So no need to be an RL expert. So don't worry. We'll go really slowly on that. So in the RL world, what people do is they have an agent-- and here, I mean the agent in RL term-- which interacts with an environment. So what does it do? It is at a given state, at, let's say, time t, it can take an action at time t. And it takes that action according to some policy. The policy is typically denoted pi, pi of theta of the action given the state. So what that policy means is simply giving you the probability of you taking an action given a state. Easy, right? And so given that the agent takes some action, it also receives some rewards. So sometimes, it's a good reward. Sometimes, it's bad rewards. So what we're going to do is to leverage that mindset for our preference tuning exercise. So let's see together how we can transpose these quantities in the LLM world. So what is the agent? The agent is the LLM. So in terms of the state that it is in, so it's simply the input that it has so far. So the action that it wants to do is to predict the next token. So it basically always wonders, OK, what is the next token given this input? And this action or this next token is among the set of tokens that are out there. So if you want-- the environment can be the set of tokens of your vocabulary. And in order to decide which token should come next, the LLM determines that using the probability of next token, which is obtained by when you do your forward pass, and when you look at the probability distribution as an output, which is basically our policy. So our policy here is simply equal to the output of the LLM given the input in order to determine the next token. So far, so good? And now, we're adding one additional thing. So I told you we're composing our preference data sets. And we want to know which output is better than the other output. And so we're going to use that for our reward. We're going to somehow use that for our reward. And we'll see how we will use that. So I'm going to just recap that part. So we have our LLM take some input, and then it wants to predict the next token. It wants to take this action. So in order to do that, it uses the probability distribution that it outputs. And then, the token that it predicts or the output that it generates then receives some reward that is going to feed back into tuning the agent. And here is the LLM. Yep. [AUDIO OUT] So the question is, wouldn't that be expensive if we do this for all pairs? Why would that be expensive? [AUDIO OUT] Mm-hmm. [AUDIO OUT] Yeah, so-- [AUDIO OUT] Yeah, the question is, what do you do in terms of how

expensive that is? So yeah, typically, people take batches, for instance. But I would not-- so it is indeed expensive. But we're going to see a little bit some order of magnitude and how that works. But you can think of this as being kind of a training procedure that could be seen as expensive as some other training procedure. There's nothing that makes this more expensive, but we're going to see exactly. So I think there are some points that you're touching correctly. There are some parts that are added compared to the regular supervised-- let's say, supervised fine tuning setting. And we're going to see that in a second. [AUDIO OUT] So the question is, does the reward that you're getting from here, I guess, powerful enough for the LLM to change? Well, you will see some expressions on the internet just characterizing this training procedure as it's nearly not as many signals as what you would get for SFT, let's say. But because for SFT, you're literally always taking a partial input and making the LLM learn how to generate the next token. But here, you're only getting roughly one signal per completion. So yeah, definitely it's more sparse. And that's why you will see that RLHF is seen as more an approach that has sparse signals. Yeah, yep. [AUDIO OUT] Great question. So the question is, do you apply a reward for each token or for the whole thing? We're going to see this in more detail, but it's for the whole thing. It's for the whole thing. But we'll see that in just a second. Yep. [AUDIO OUT] So the AD and-- oh, right, so the question is, what is the AT and ST. So just as a reminder, ST is the state that you're in, and AT is the action you want to take. So in the context of an LLM, the state that you're in for an LLM is the input that you have so far. And then the action is which token you want to generate given that input. Yep. Cool. Yeah, so far so good? OK, perfect. So now that we know a little bit what a mental model could be for, I guess, LLM-based RL, I just want to highlight once more that what we're trying to achieve is learn how to align this policy with rewards. So we want to learn theta, and theta is the parameters of our LLM, such that pi of theta aligns with preferences. And that's where RLHF comes into play. So RLHF stands for Reinforcement Learning from Human Feedback. And it is typically composed of two stages. So the first stage is you figuring out how to distinguish good output from bad output. So here, all the preference pairs that you collected are actually used for you to learn what is good, what is bad. So the input here is the concatenation of the prompt and the response. And the output is a score. So what you want to know given a prompt and the response,

how good that is. And then the second step is the RL step, the Reinforcement Learning step. And this is where you use the rewards to align your model with the preferences. So here as input, you have the prompts. And what you somehow want to do is to be able to generate y hat that is more aligned with the rewards. And by the way, I just want to call out one thing. So RLHF is Reinforcement Learning from Human Feedback. The Human Feedback part refers to the labels on which the reward model is trained. So if there are preference pairs are based on human ratings, then we're relying on human preferences, then we're in RLHF. Because you will see out there, there is also RL, let's say, AIF, a Reinforcement Learning from AI Feedback. And that one is relying on non-human preferences. So yeah. Cool. OK, so now that we know what RLHF is and we know that there are two steps, we'll go through the first step

Reward model

Naturally. So here the idea is we want to construct a model that knows which output is good, which output is bad. And of course, here, what we want is to not only consider the output, but also the input, because you need to somehow contextualize that answer. So let me go with our favorite example. So let's suppose you have the following prompt. Suggest a new activity I could do with my teddy bear. So what you want is to have a reward model that-- here, we note RM, --that tells you that the answer that we rewrote into the good answer is good. And we want to somehow have that model that tells us that the output that we constructed-- or that we did not construct, the output that we saw was bad is bad. So we want to somehow have a model that takes in the prompts and the good response and say it's good. And we want to somehow have a model that takes the prompt and the bad response to say it's bad. So now the question is, how do you construct such a model? Well, in order to do that, we are Bradley-Terry formulation using a formulation that's called the Bradley-Terry formulation. So that one is an important formula. So we'll just stay here for a little bit. So what it says is that the probability-- bless you-- to have an output yi be better than an output yj is equal to an exponential of some score, which is a score with respect to i over the exponential of that score with respect to i plus the exponential of some score with respect to j. So that is

called the Bradley-Terry formulation. And this is the formulation we will use to build our model. So here, it's also equal to sigma of ri minus rj. So who knows what sigma is? [INAUDIBLE] Yes, exactly. So it's a sigmoid. So just as a reminder sigmoid is 1 over 1 plus exponential of minus x. So this is how the graph looks like. So when it's minus infinity, it tends towards 0, and then towards plus infinity, it tends towards 1. So in other words, if i is better than j, what we want is for the input to sigma to be as high as possible, because you want the probability to be as close to 1. So you want to somehow have ri be high if the output i is good and somehow rj to be low, if the output is bad. So far so good? So here, what we want to do is to somehow train a model that is able to output scores ri and rj using this formulation. So this formulation involves two quantities, because we have a pairwise data set. So far so good? So it will become more clear in a second. So here for training, the idea here is you have some model that you initialize, and you input on the one hand your prompt x and your winning output. So I'm saying winning, so it's y hat w. You put it into the model. It produces score r of x and y hat w. And then, you have a second output, so x and yl, y hat l, which is the losing output. So you put it into the model, and it has a second score. And now, you somehow want to have a loss function that takes into account these two scores. So based on this formulation, do you have some suggestion as to which loss function to use? So here, our loss function would be pairwise. Yep. Yeah. So the proposed answer is a binary cross entropy. So can you explicit that a bit more? [AUDIO OUT] Mm-hmm. [AUDIO OUT] OK, so I guess, in that case, what would it be? [AUDIO OUT] Yep. [AUDIO OUT] Yep. [AUDIO OUT] Yeah, exactly. So I guess, the-- yes, great answer. So I guess, your answer is having some negative log likelihoods of this quantity, which is-- so the cross-entropy can be seen as kind of a special case of this. So that we make sure that we get this part-- so in order to just get that formulation, which you mentioned again, I guess, one idea that we can have is given our data and given this formulation that we saw, the Bradley-Terry one, to find parameters theta that maximize the probability of that data happening, which basically, leads to what you mentioned. So here, I'm just going to write what that means, so that just we can be all aligned. So here, it's-- so let's suppose you have a preference data sets of, let's say, winning example associated with losing example. So you have a bunch of pairs like this. So let's suppose that these pairs, they happened independently of one

another. So what you want is to find parameters that somehow maximize the probability of you seeing those examples. So here, what that means is we want to somehow how maximize the products of, let's say, these n preference pairs, which is the probability of some output w being better than some output [? l. ?] And we saw that with the Bradley-Terry formulation, so we have this formulation, which is basically the product of [? y1 ?] to n of these sigma of this reward, which is basically a function of the input prompts and the output. Like this. So what I'm doing here is to just reconstruct the loss from first principles. So whenever you see a product of probabilities, the first thing you need to think about is-- yes, log. Because this can become very small, so it can cause instabilities. So if you take the log, if you want to maximize the product, it's the same as maximizing the log of that. So if you take the log of that-- so let's suppose I'm taking, let's say, the log of that. Let's say I'm taking the log of that. So it's basically equal to the sum of the log of sigma of r of the winning minus the one from the losing. So we want to maximize that. But people in ML, they like to minimize things. So we're going to take-- like, have a negative in front, so maximizing plus the sum of the log is the same as minimizing minus the sum of the log. And so this will be our loss function. Does that make sense? And this is exactly what you mentioned. And typically, we write the expectation. And this is our loss function. So our loss function is minus the expectation of log of sigma r and x and y, y hat w minus the reward of the losing. Sounds good? So here, the loss function is pairwise. But the reward model, is it pairwise or is it pointwise? I guess, do you need a pair to make a prediction or do you need just one? Do you need a pair? Pair? Well, that's the beauty of things. You don't need a pair, so you're just training it pairwise, but it's actually pointwise. So I think that's one thing I realize. I look at this loss, and this is a [INAUDIBLE] So you're training in a pairwise way. But at the end of the day, you have a reward model that takes in one prompt and its output and just outputs one number. And we saw that it will try to output a high score for winning examples and then a low score for losing examples. So this is our reward model. And I'm looking at the time, I'm actually not on time, so I'll just move on. So typically, here, what you would need is a set of data, which is typically in the tens of thousands or maybe even more. And here, the label would be the preference rating. The preference here should come from humans if we're talking about RLHF. And in terms of model, where you have a bunch of different choices. So as we now,

models that are decoder only that predict the next token are very popular. So you could very well take such an LLM and just have some classification heads at the end of your sentence that you're taking into consideration and then use that to predict the reward. Or if you remember, we've also seen encoder only models in the beginning of the class. So for instance, BERTs. You could typically also project the embedding of the CLS token. It could very well be an option. So typically, what people do nowadays is take the LLM route because everything is an LLM these days. So it's like decoder only. You just put a classification head. And so people have also come up with benchmarks to evaluate how good you're doing. So we just put a reference here RewardBench in case you're interested. It's a pretty popular one. But yeah. So at the end of this, you get a reward model that takes in a prompt and a given response and gives you a score. Yep. [AUDIO OUT] Exactly. So the question is in some tasks, the human preference would be something, but another task, it may be something else. So typically, those rewards, they are with respect to a given dimension. So they can be, let's say, is the output useful or is the outputs are not friendly? Is the output safe? So all of that are different dimensions, so they can be different reward models. You can also have some holistic score as well. But yeah. So you need to define a dimension across which you're actually quantifying how good your response is. So the ones that I mentioned are the common ones that you'd find. Another thing that we say as well, while we're at it, so human ratings are very sensitive to the guidelines that you're also exposing. So we're not going to go into details here, but one important aspect of human preference data is to make sure that the guidelines you're telling your readers are as objective as it could get. I mean, sometimes you cannot have them be super objective, but you have, I guess, the task of just making sure they're clear enough, so that your human preferences, they're not noisy, because they can be noisy, but that's also one challenge. Yeah. [AUDIO OUT] So the question is, is the reward here a regression or a classification? So let's look at the loss function and here. So well, it's hard, I guess, I will say because the reward can be something that can be interpreted as a score, but we have a probabilistic formulation. So I'll probably frame that as a classification task because you have preference data. Is it better or worse? So it's 1 or 0. But at the end, you use the rewards. So it's not, I guess, purely a classification task because at the end, you have this. But one thing to note is that the scale that

these rewards are at is typically something that is scaled itself. So at inference time, you have some normalization procedure that normalizes that across your batch. But yeah, I would probably characterize the formulation more as a probabilistic one I would say. [AUDIO OUT] Yep. Yep. [AUDIO OUT] Yep. So the question is, do we normalize the score? So there are many different methods to, I guess, normalize things. We typically do. We typically do normalize. So I guess if you're in the regression setting, you're trying to predict some score on a given scale, which you're not doing here. So it's free form here. So yeah. So there is some rescaling that happens. Yeah. You had a question. [AUDIO OUT] Yeah. The question is, can you tell us more about what the output of a reward model is? So we're going to see that a bit later, but you can think of goods outputs as being, let's say, 1, bad outputs as being minus 2, minus 3. So it's basically on the continuous scale if you want. We're going to see an example in a little bit. So hopefully, this can-- do you have a question? Perfect. We're going to see an example. So hopefully, that will be clear. Cool. So what we did is train a reward model.

Reinforcement learning

Now, the second step is to use that reward model to align our model, and by model, I mean the LLM. The LLM that went through the pre-training stage and the SFT stage is the model that we want to align with human preferences. We're going to do that using reinforcement learning. And we're going to do that using the reward model that we just constructed. So the reward model here is the model that we obtained in step one. And it allows us to distinguish good outputs and bad outputs. So here is a general recipe of how you would align your model. So first, you would take your prompt as input. Your LLM will generate a completion. So here, a completion means a full response from the model. So by the way, completion, people use also the term roll-outs. So it generates a completion, a rollout, a full response. And that full response along with the prompt goes into the reward model. So as we saw, the reward model right now knows if it's good, if it's bad. Let's say, right now, it's bad. So in practice, it does not generate the thumbs down. It generates some kind of score. So if you want minus 2 let's suppose. So we take that reward into consideration. And then what we do is we tune the LLM with this information.

So it's more complicated than that. And we'll see how we go about doing this. But this is the general idea. So just as a reminder, the reward model is the model that we trained as step one. And it is a model that is frozen. We're not training the reward model. The reward model has been trained. The model that we're training is the LLM. So I think that's an important point to note. And our goal is to optimize for higher rewards but then without going too far from the initial model. So I think the optimizing for higher rewards, I think, everyone agrees with me. But I just said a second statement, which is but we don't want to go too far from the initial model. So why would that be? Why would we want to not deviate too much from the base model? [AUDIO OUT] So one suggested answer is it will catastrophically forget what it has learned. Why would that be a problem? [AUDIO OUT] Great. Yeah. I think it's a great way to put it. So the suggested answer here is you have all this knowledge in your initial model, which is pre-trained and then instruction tuned or tuned and you don't want to move away too much. This is exactly it. So that's one reason. What could be another reason? So definitely one. It's very, very good reason. What could be another reason? Yeah. [AUDIO OUT] Yep. So overfitting on that data. So can you tell me more? [AUDIO OUT] Yeah. Great point. So the second suggested answer is you can overfit on data that is not super clean. And I will go a bit more into detail on what that means. Yeah. You wanted to add? [AUDIO OUT] Yep. Yeah. Exactly. So I guess your two points are, I guess, along the same lines, which is the reward model can be noisy. So that's exactly it. So it's a phenomenon that's called reward hacking, where if you are trying to optimize too much for the rewards, you're basically assuming that the reward is exactly quantifying what you want. But it may not be the case, because the reward model itself is imperfect. So just to make sure that everyone gets the point, I'm going to illustrate that with an example. So let's suppose that I'm giving a lecture. And my objective is for the lecture to be as informative as possible. So I can take the choice to have something that I can quantify to know if what I'm saying is informative. So let's suppose if my reward is how loud the claps are at the end of the lecture, if the clapping is a lot or not. The problem is if I optimize too much for the volume of clapping at the end, it may be the case that if I found out that you all like jokes and I start making jokes and then you clap at the end, my reward is maximized. But my objective is not fulfilled, which is make the lecture informative. So in practice, this reward hacking phenomenon

is something that leads your model to optimize too much on a metric or a score that is imperfect. So it can very well maximize the rewards but not fulfill what you want. And this is exactly what the idea behind reward hacking is. So yeah. And another reason by the way. So it's one reason is because your base model is already great. It knows a lot of things. Second one is because the reward model is imperfect. So you have this reward hacking phenomenon. The third reason is also because you can have some training instabilities. Anyways, we have a bunch of reasons why we don't want to deviate too much. And so in this setup, you typically have more observations than in the reward model case. So you have typically at least 100K observations. And the label that you get here is from the reward model. And so you start with the model that you obtained after SFT. So that's the one that you trained. And the loss function will be something that will try for you to maximize the rewards but also to not have you deviate too much from the base model. So we're going to see exactly how we can do that. But that's the idea of how our loss function will look like. It will have these two parts. Sounds good? And as we saw, so we don't want it PPO to deviate too much because of reward hacking and instability and so on. So in order to do that, you may have heard a very popular RL algorithm that's called PPO. So PPO stands for proximal policy optimization. So the reason why it's named proximal is because we do not want our model to deviate too much from the base model. And we're going to see how we do that. So this is roughly how our loss function will look like. So we'll have a part that is a reward model that we want to maximize. And then some other part that is around measuring how far the distribution of the policy that we're training is to the reference model or the base model. So here, the reference model would be your SFT model. So you have these two parts. Does the formulation of the loss roughly make sense so far? That makes sense? Cool. OK. KL divergence. So I'm just talking about KL divergence, but I don't want to just assume you know what that is. So the KL divergence is some measure. I don't want to say distance because distance has a lot of connotations to it. It's not a distance, but it's some measure of how far two probability distributions are. So you take a probability distribution, let's say, P and another one probability distribution Q. And you compute this formula, which is the sum of p i log of p i over q i. Just gives you a number of how distant or how far these two probability distributions are. So you want to minimize the loss. So I guess you want to minimize how far they are. Does that

make sense that we're using this? Yeah. OK. So just one little question. So OK. Is the KL divergence positive or negative or what can you tell me about the KL divergence? Have you seen the-- yeah. It's positive. Why is it positive? [AUDIO OUT] Yeah. But why? So maybe we can come later but great, great. I think the result is here, but I guess I want to why. Yes. Exactly. So yeah, if you use Jensen's inequality-- because when you look at this formula, I mean, luck can take positive and negative values, it's not super clear. But yeah, the way you prove that it's always positive or equal to 0 is with the Jensen's inequality. And so yes, this quantity is equal to 0 if and only if P and Q are equal. Cool. So with that, we're going to go a little bit more in depth into the loss function. So I mentioned that the loss function tries to maximize for rewards but also make it so that the model does not deviate too much from the base model.

Advantage, value function

Well, there is something that I've lied a little bit about too, which is we don't want to maximize rewards. We actually want to maximize a quantity called advantage. So what is the advantage? So you can think of the advantage as being how much better is your output compared to what you would expect. So people typically use advantage to make the whole process be, I guess, more stable, the training more stable. And we estimate that advantage with a function called a value function, which we will see in a little bit. Did you have a question? No, no question. So we have two quantities. Right. We have the reward that we saw previously how to compute, which is given a prompt, given a completion, how good is this prompt and completion? And then the value function is another estimation, but this estimation is not at the completion level. It is actually at the token level. So what we're doing here is we're taking the prompt along with the partial generation as input. And we're trying to estimate what the reward would be if we were to continue to generate the output following the policy. So I'm just going to repeat this one more time because it's charged in meaning. The value function is a token level estimation of the reward that takes in a partial input and predicts the reward. So that predicts the reward if you were to follow the policy in your generation. So this value function is something you will see in papers as something that people have to train when they do PPO. And it's typically because people need to have this value

function when they compute the advantage that I mentioned above. And this value function is typically trained jointly with the policy. So in other terms, you have your LLM, which is initialized as the SFT model, which generates the prediction. So what you typically do is also have some value head that also-- so this time, it's not a classification problem. It's a regression problem. You try to estimate the final reward if you were to continue to generate according to policy. So this is typically something that you also need to train jointly with the policy. And I am intentionally not going too much into detail as to exactly how that advantage is formulated. It is just beyond the scope of this class. But in case you're interested, I would highly recommend reading this paper, which is mathematically involved about the generalized advantage estimation methods, which people typically use for this. So it's this high dimensional continuous control using generalized advantage estimation paper. So if you're interested, just feel free to take a look. But this is typically the method that people use to estimate these advantages. So now, we're going to see in a second where those advantages are used. But just as a high level, does that make sense to everyone what we're trying to achieve here? [AUDIO OUT] Yeah. So in other words, what we want is to maximize the rewards but not deviate too much from the base model. But what we want is to actually find a way to make the rewards a bit more relative as to what you would actually expect. So you want to compare the rewards that you're obtaining with an average response if you want. The idea is, OK, if you are having a great reward, what would be the reward in the average case? And you want to maximize that. And the reason why people use these reward minus baseline is because it reduces the variance of these estimates, and it just makes the training just go faster typically. So that's why people use that part. But in order to compute the advantage, what people use is a method called generalized advantage estimation, which you can think of it as a big formula with some hyper-parameters that is in the paper below.

PPO variants: Clip, KL-penalty

That's the general idea. Cool. OK. Looking at the time, so we're going to look at two typical variants of the PPO loss. The first one is called PPO-Clip. So the idea here is to prevent the updates to our model from being too large from one

iteration to the next. So what you have is this loss formulation, which is the minimum of the ratio, and we're going to see what that is, times the advantage and some clipping of that ratio between two bands, which we're going to see in a second, times that advantage. So before we go into details, I just want to call out a few things that may seem like it's confusing because it was confusing to me. So first of all, that formula is expressed as L is equal to, but it's actually not something we want to minimize, something that we want to maximize. So what people typically mean by L is loss. So yeah, just know that we want to maximize that. We don't want to minimize. The second thing is we're introducing a term called R of theta. This is not the reward model. So typically, people use R to say the reward. But here, it's not the reward. It's actually the ratio in the probabilities between our policy and the policy from the previous step, which is denoted pi old, pi theta old. So it quantifies how different your policy at the current stage is to your old policy. So here, when we say, old, it's not the SFT model. It's not the SFT model. It's the model at the previous iteration during the oral stage, because here what we want is for the policy to not have updates that are too wide. So far, so good? Yeah. [AUDIO OUT] What is A? A is our advantage. So you can think of it as this complicated formula that's a function of the reward and the value function that just tells you how good your piece is basically. So this complicated formula, we're just going to see why it makes sense. So if the advantage is positive, it means that we somehow want to reinforce whatever was generated. And so here, our objective function will look like this. So if A is positive, then this L of CLIP will look some linear function as a function of R because it's R times A. So it's a linear function as a function of R. And then after 1 plus epsilon, it will just be flat. So why do we want such a shape for our graph? Well, we need to think again about what R means. So if A is positive, meaning it's something that we want to reinforce, we typically want the probability of whatever happened to be increased. So we want R to be higher, but we don't want it to be too high. So that's why we have this clipping mechanism. So that's why we have this clipping here. We want to, I guess, maximize L. So we go towards increasing R, but we don't want to increase R too much. So that's the rationale for this. But when A is negative, it means that somehow what we've done is something that we don't want to do as much. So in order to maximize L, what we want is to actually decrease the probability of just generating what we generated. So we

want R to be smaller because it also increases L, but we don't want it to be much smaller. We don't want too big of an update, which is why we have this clipping mechanism here. So in other words, this super complicated formula can be just intuitively explained with what I just mentioned, which is if you have positive advantages, you want to reinforce the fact of generating this output because this is something that the model likes. I mean, by model, I mean the reward model, but you don't want it to update it too much, which is why you have this clipping. And same when the advantage is less than 0. I guess you want to down weight the probability of this output from happening. So you want to have a smaller R, which is your ratio between the policy you're training with respect to the old one, but you don't want to be too small either, again, to not make too big of an update. Does this make sense? Does this roughly make sense? OK. Roughly makes sense. Cool. I will just recommend just rereading this formula, just remembering what I mentioned. And hopefully, it will make a bit more sense. But yeah, happy to answer any questions if there are any. Yeah. Oh, yeah. Yeah. [AUDIO OUT] So the question is, what happens where it's not-- it's the same as ReLU. You will basically use the same methods. So yeah, it's the same thing. Yeah. [AUDIO OUT] So great question. The question is, why are you using from the previous iteration and not the base model? Because actually, we don't want just to not be too far from the base model. We also want our updates to be not too extreme from one iteration to another. And this is to improve with stability of training. So this is just like another additional constraint we want. [AUDIO OUT] Question is are we considering the base model? We will consider it. And I will tell you exactly. But in this formulation, we are not. In this formulation, we are not. But in the formulation we're using typically for LLM based RL, we are and we will see how. Yeah. [AUDIO OUT] Yeah. So the question is, how do you quantify R? What is R? What does it mean? So it's typically the probability of a token happening with respect to the inputs. So you can think of the probability distribution that you get at the output of your LLM. So you would use that. Cool. So this is the first variant that the PPO paper mentions. So it's called PPO-Clip. So the second variant that we have is the one that uses KL divergence, which is called KL penalty. And this one is a function of the ratio times the advantage minus the KL divergence between the old policy, which is the previous iteration, and the current iteration. So you had a great point, which is why are we using the old? In fact, old means previous RL

iteration. But nowadays, people use the reference. So it's just a change-- because this paper has been published in 2017, where in 2025, LLMs, they came more into something that was popular in the 2020s. So yeah, there's just been this difference in what you put there. So yeah, it's indeed the reference that you would typically put in a KL divergence. The other thing that I will say is that nowadays, when someone wants to train a policy using PPO, they would typically use the KL divergence, and they would also maybe mix it with the clipping thing with the old iteration. They will basically mix the two. There are many different formulations of what the loss can be. So what I'm saying is not always true. But this is what people sometimes do. They sometimes mix the two. Cool. So I talked to you about PPO and how it allowed you to maximize rewards, maximize advantages by not making your model deviate too much from the base model and also from the previous iteration. But the problem is that for PPO, you need a lot of models. So you need your policy, you need your LLM that you're training. You also need the value function, which what I mentioned is used for the advantage estimation. You also need the reward model of course. And then you need the base model. So the base model is frozen, so just the SFT model just to be able to compare with the current policy. So it's a lot of models. So now, the question that you may ask is, do you really need to have all these models to perform well? And the answer is maybe not, maybe but maybe not. And nowadays, there is a bunch of other methods that are becoming more popular. So you may have heard in GRPO in the context of reasoning models. So we will look at GRPO in more detail next week for lecture 6. But just know that there are many variants. So PPO is just one algorithm that has been very popular. And even within PPO, there's been many variants. And nowadays, there are many other RL algorithms that are being used. Yeah. And yeah, if you want to just anticipate next lecture, you can also take a look at this paper from the Deep Seek team. So Deep Seek Math limits of mathematical reasoning in open language models that introduces the GRPO algorithm. But if you don't, it's fine. We're going to talk about it next week.

Challenges

So I also want to talk about some challenges when it comes to RL-based approaches. So the first one is you need to have this two-stage process. You

need to first train your reward model, and then you need to use the reward model to train your policy. But the problem is you do step one, you do step two, and then you realize, oh, wait, step one has a problem. Then you need to redo everything. So there is a bunch of dependencies that can really make your life harder. And just the training recipe is not the simplest. So that's one downside. The second downside is about the number of hyper-parameters that you need to tune. So we saw beta from the KL divergence formulation. So that's one. We have epsilon from the PPO-Clip version. That's another one. The generalized advantage estimation, we have not seen the formula. But there's, I think, at least two hyper-parameters that I can think of. Anyways, you have a bunch of them. So of course, some hyper-parameters perform better than others. And if you need to retrain, then, well, you need to do everything again. Then you have some instability challenges. So you can constrain your iterations to not be too wide to control for that but sometimes, it's not sufficient. So in terms of the metric that you can use to monitor the training, well, what is the metric you would use? So we're trying to maximize rewards in some sense. So typically, the metric that you would use to monitor your training would be the average rewards. But this is not a great-- I mean, this is a way to monitor your training, but this is not the best way necessarily. So at least when it came to pre-training and SFT, you had this cross entropy loss that you were monitoring that was really telling you how your model was able to just reproduce the behavior you were trying to make it outputs. But yeah, so this is another challenge. And then the other thing is in the RL world, in order for you to know which completion you should not do and which completion you should do more, you need to have some diversity whenever you generate. So if during the RL training loop, every time you have a prompt, you generate something, and then you retry, and you generate something else that's too close, well, it's not great because you're not exploring the set of possible completions. So this is another challenge you need to have in mind, which is you need to force your model to do some exploration to see, I guess, what can be good. And then the last point is, I mean, a lot of you were not familiar with RL. I was not, and it's not super clear why we would necessarily need RL to do this stage.

On-policy vs off-policy

Also, I will say one thing. So this RL-based training, you will hear a lot the term on policy training. I just want to tell you what that means. So what is different between this and SFT is during SFT, we had some prompts and some response that we wanted our model to generate, to mimic. But here, what we're doing is at each iteration, we're asking our model to generate an output, and then we're using how good that output is to update our model. So there is a core difference between the two, because in here, what we're doing is we're asking the model to generate something, whereas for SFT, we're just using some data that is not necessarily generated by our model. So on policy training refers to training that involves the model generating from its current policy and optimizing on that. And this is in contrast with off policy training, which is relying on generations that are not done by the model that you're training. Does this make sense? Yeah. So this is a term you will see a lot, which is why I'm telling you. And yeah, so PPO is an on policy algorithm. Yeah. [AUDIO OUT] So the question is, why don't we do SFT on this preference data? Well, SFT, you tell the model, you should generate this given this input, then you should generate this given this input. But you're not telling it about things you should not generate. Let's suppose you have something that you don't want to happen, unless you rewrite that into perfect sentence, there is no way for you in the traditional SFT framework to tell your model to not generate that. But that being said, it's a great prompt because later on in, let's say, 10 minutes, we will see a supervised way of doing what we did with RL. But yeah, it will be in about 10 minutes. But apart from that, does that roughly make sense?

Best-of-N

Yeah. Perfect. So in the next few minutes, we will see a method that is actually used more often than not. And this is for people who have a reward model but that don't want to do RL. So why do you not want to do RL? Because it's expensive. You need a lot of models. So to your point, it's indeed expensive because you need all these models. And sometimes it's just too hard to tune that training. So you have these methods called best of N or BoN. And the idea here is to leverage the reward model that you obtained in order to choose the

output that you will return to the user. So the idea here is you have a prompt, and you tell your models, OK, you actually want to have several generations. So you generate, let's say, 4 times, 5 times. And what you do is you give a score to each prompt completion, and you just return the best one, the top rated one, which is why it's called best of N. So you have N completions. You rate all of them, and you just return the best one. So you have your prompt. So our favorite example, suggest a new activity I could do with my teddy bear. You put it into your SFT model. So you're not training your SFT model. It's as is. And you're just generating, let's suppose, three completions. The first one is a great answer. The second one is no, don't spend time with the teddy bear, which is not a good answer. And then the third one, I think, is also a good one, take your teddy bear to a picnic. You put all of these into your reward model. And this is where I wanted to respond to you, what is an output of the reward model? So it would be a score like this. So for instance, the first one would be a pretty good completion, so it's 0.8. The second one is not good at all. Let's say, minus 2. And the third one is, let's say, so-so, so in between. So the best of N method would select the top rated one, which is the first one, and this is the one that you would return. So what is one problem with this approach? [AUDIO OUT] So the answer is if the model is bad, then it's going to be bad. That's fair. But I guess if you make enough completions with, let's say, a high enough temperature, you will have some diversity. So it's a fair point, which can be a problem, but let's suppose it's indeed a problem. Let's suppose it's not a problem. What is another problem that you may have? Yep. [AUDIO OUT] Yes. Yes So the answer is you query the model many times, which is indeed the problem. So you're at inference. What you want is to not spend a bunch of money and compute. But here, what you're doing is complete your prompt multiple times. And this is indeed the main challenge with this. So you indeed skip the RL training, but you're basically pushing all the work to your inference, to the inference stage. And this is just making everything so super costly. So when can it be bad? So if, for instance, you're in a case where you need to serve your model to, let's say, a bunch of traffic, it may not make sense for you just from a cost standpoint. So I would say, before taking that route, it may be good to just assess what will be your inference traffic, what would be your training cost, and determine that from there. And yeah, I think that's pretty much it. Do we have any questions on that part? Oh, yeah. Yeah. [AUDIO OUT]

That sigmoid would be softmax. Are you talking about this one? So in that algorithm, you're assuming that your reward model is trained. So the best of N approach is you do this training. So I guess the sigma here is pairwise because you trained your model in a pairwise fashion. But in the best of N case, what you do is you just use your reward model to score N times, and you just pick the highest one. Yeah. [AUDIO OUT] Yeah. The question is, but what if all the responses are bad? This is definitely a fair concern. And this is indeed a concern. So your model needs to be at least a little bit good. Yeah. Exactly. Yeah. Yeah. Yeah. [AUDIO OUT] It's a great question. Do you have any constraint on the scale? Well, depending on how your scaling is done, you actually don't care about the scale as long as you take the best one, the highest one on. If you think about it, let's suppose if you scale it, let's say, with a normal score, the highest one will still stay the highest one. So in this case, you actually do not care about the scale. [AUDIO OUT] Yes. You also don't care about that. But typically, the scaling matters when it is coming into play in a loss function, which is the case for RL. And so that's where people use some scaling mechanism. That's the part that matters.

DPO

Cool. And with that, I'm going to give it to Shervin. Thank you, Afshin. So for this last part, we're going to talk about DPO. But I'm not going to tell you what DPO means just yet. We're just going to first listen to the complaints we had in the RL world and then see what we could do about it and then how have folks remedied it. So the first thing that I'm going to repeat from what Afshin covered is that in the RL formulation of PPO, you have to carry, during your loss optimization, a bunch of model weights. So when you look at this loss function, you have, first, the policy of the current model you have to optimize for, the one of the old model or the reference model. And then you have your advantage which hides the reward model, as well as the value function that we discussed. So all this four, which is a lot. And the second thing is that, let's say, you go with the approach of best of N, you don't do RL, you just generate multiple times and then pick the best completion. As Afshin mentioned, so you have this latency and cost issues. And just for the sake of thoughts, for a thought experiment, let's suppose we had infinite money, would it be still fine? So let's

suppose you generate all of these in parallel. So you would still need to wait for the maximum time of the generation of an answer to give you a resulting answer. And then if you look at the distribution of latency, usually it's some shape that is not centered on a single point. And when you look at the probability distribution of the max of, let's say, N answers, it will tend to be shifted to the right. So let's suppose you have infinite money, infinite compute, you still have to wait more than you would have on a single pass. Do these concerns make sense? And OK. So there comes the question that someone was asking, why don't we do supervision all this time? And this is the route that DPO folks have explored. So DPO stands for direct preference optimization. So instead of doing these expensive two-step process of, first, finding some reward model and then iterating on the model's weights, it optimizes a single loss function that directly optimizes for the model's weight. And as you can see, there is no reward involved anymore, so no R. And you have a way to formulate what you care about as a function of your preference pairs. So in that loss function, all you have is a bunch of variables and then the sigmoid functions. But within the parentheses, you have the probability of a given completion to happen. And you have this subtraction operation that compares the probability of the winning completion to happen versus the losing one. So as you can see, it's a direct expression with respect to these preference pairs, which is pretty nice. And if you look at it a bit closer, you will recognize what afshin just mentioned regarding Bradley-Terry formulation because you have these expected value of log of a sigmoid. And then within that sigmoid, you have a difference of terms, where you can recognize the equivalent of what you saw as a reward term. So I know it's a lot to see at once. The formula is not super pretty. Do you have any questions? And then we're going to discuss in a bit what that beta is and also how we obtain this formula in the first place. And one thing I will mention before going to that is that the paper is titled your language model is secretly a reward model. And the reason for that is that the reward placeholder that you could identify from this loss function is expressed as a function of your policy. So you don't have any r's in there. But when you express the equivalent of a reward, you find the expression of your model directly. So which is pretty interesting insights. OK. So but now, let's think together about how we got there in the first place. So you recall from a few slides ago the PPO objective that we had. So we wanted to maximize rewards

and minimize the distance of the obtained policy with respect to the reference one, which is this trade-off between maximizing rewards and KL divergence term. And then you have this beta coefficient that appears there that controls how much you want to penalize going further from the base model. And what this paper did is that it wrote down this formula and then solves what would be the optimal solution. So it expressed the optimal policy pi star. And when you derive this expression, so you get it as a function of R. So that's another term that was in the objective function. And so all these steps have no extra assumptions. It's just a bunch of derivations that was done. And what you can see here is this Z term that stands for a partition function and that just normalizes things. But it's not something that is new. It's a function of the terms that were already in there. So when you rearrange these terms, you can equivalently express r as a function of this pi star. So it's just rearranging terms between these two expressions. Nothing too fancy. And then the key part of what this paper did is recognize this term as some reward and then express it as part of a Bradley-Terry formulation. So you remember you had this probability of a completion, of a winning completion being bigger than a losing completion. And you had this sigmoid of the difference between the two r's. So basically, they took this formulation again, and they plugged in the expression they had obtained for the optimal reward, which we had seen was a function of the policy. So at the end of it, you plug in things. You have no more rewards, just a function of the other parameters. So you have this beta and only the policy in there. And the last step is the same one as Afshin was mentioning, once you have this probability that you want to maximize, you can turn it into a loss function. And this loss function is what this paper proposes to optimize in order to tune policy weights in order to align it with this preference data. So I simplified a lot the quite heavy steps of this paper into these few steps. Do they make sense? And the beta that we see here is the same beta as we had seen in the PPO formulation, so typically taken around 0.1 if you want some order of magnitude. So beta is a hyper-parameter, and all you want to optimize for is the policy. And then the preference pairs, you have them as inputs. Makes sense? OK. Great. So now, let's compare what this method gives with respect to RLHF. So RLHF is this two-stage training process where you first fit a reward function and then use it in order to do the policy update steps. So Afshin was mentioning it, it's quite heavy. We were mentioning it just before, you need a lot

of models in memory and then training stability is a concern. You don't have direct supervision. You rely on the policy of the current model to generate the next updates. So there is a lot of complexity in it. And as opposed to that, this DPO formulation gives a supervised way to think about preference tuning, where you just take this preference pairs, and you directly fit a loss function on them. And yeah. And instead of four models, you just need two because you had this pi of theta and then a pi ref, which was your base SFT model, and you don't need any other model copies. So now, you might want to ask if it's much easier, why is not everyone just simply using DPO? So it's not as easy. In practice, there are some pros and cons to each method, and the paper that is listed below gives a very interesting study of the bench-marking differences and the steps to get there. So just to get the main lines, overall, PPO performs better than DPO. And one great thing about DPO, which is the supervision, the direct supervision, comes at the cost of sometimes fitting something that is not the exact distribution that the model has seen as training time. So this paper talks about it a bit. If you train via SFT on the preference related data, you could get better performance. But this shift in distribution is a challenge intrinsic to DPO, because one thing that the DPO method enables you to have is just to not worry about that reward modeling at all and just get some preference data set out there. But the issue with doing so is exactly that distribution shift. So you could either train SFT on it or donate it yourself and then rate it. But that is yet another cost that you would need to pay. Great. And yep. Any questions? Yep. [AUDIO OUT] So the question is you are optimizing the parameters with respect to the base model. So yes, but it's not the base model. It's the current one. You have a base model that will stay fixed in the iterations. And it occurs in the loss function. And you're indeed updating parameters. And that will be like the preference tuned one. Yeah. So you have two weights, two set of weights, one frozen, one that you update. Yeah. OK. Great. Any other questions? [AUDIO OUT] Yes. So the question is instead of the reference model, could you use another model, another large model? So I think that's an interesting point. I guess that would be a different algorithm. And you might lose a bit the meaning of what you're trying to do because this preference tuning stage is trying to align your responses to what you prefer. And you want them to align with respect to what you had learned so far. So if you start from a different model, and then you preference tune a different probability distribution, you might be

doing things that might not make as much sense from a loss standpoint. But I guess the direction-- I mean, I cannot tell a blanket answer there. Maybe it could give interesting behavior, but my intuition would tell you that the point of all of this is to start from the last train checkpoint that you had and then further preference tune the distribution of your completions. So the suggestion was if you change the reference model, that could be seen as another SFT version of your model. Yeah. I guess so. Great. And now, my favorite part of today. So do you all remember the completions we had obtained on our favorite prompts? So can I put my teddy bear in the washer? So if you remember last episode, we had said after instruction tuning, the response was, no, it might get damaged. Try hand washing it instead. So does anyone have any complaints regarding this answer? So I can give a hint. Teddy bears, they indeed should be hand washed. So it's factually correct. But let's suppose it's factually correct. What else do you not like? I mean, could you not like about this answer? Yeah. So the suggestion was it's asking to try it instead. And I guess a broader point is it's a bit too rough. It doesn't sound super friendly. And then someone who asks this question might love teddy bears, so you might want to tell them about it in a gentle way. And this is exactly what preference tuning stage is about. We don't want to learn new facts. We want to focus on existing distribution of completions and align what the model should return with respect to what the human prefers. So on this intuition, the preference tunes answer could be a gentler one. It's better not to. Your teddy bear could get hurt. A gentle hand wash is safer, so it's conveying the same points but in a tone that sounds much better to the ears of the person who asks. Great. Any questions on the DPO part? Yeah. [AUDIO OUT] The question is, what are the trade offs and then which one is used in practice? So I think it all depends on your compute budget, on how much you want to worry about performance versus babysitting your training process. PPO, like the RL part, is very hard to tune right. It will give the highest results, but you might get almost at that level with much less efforts. So I guess if you want to get a quick preference tuning that is shown to show good results, but maybe not the best results, then DPO is your friend. And if you're an expert at reinforcement learning, you know what you're doing and you want the every last bit of performance, then PPO might be a better way. Amazing. And with that, thank you very much.